

ECE 3301L Section 5

Fall 2025

Final Project:

Autonomous Stabilization System

For A Coaxial Helicopter

Names: Jonathan Dau & Kevin Davis

Demo Date: 12/5/25

Report Date: 12/5/25

Table of Contents:

Introduction	?
Overall Goal	?
High Level Design	?
Rationale	?
Hardware/Software Trade-offs	?
Hardware	?
Microcontroller	?
Helicopter	?
Stabilizer Motors	?
Gyroscope/Accelerometer	?
Software	?
Stability System	?
Helicopter Take-Off/Landing Routine	?
Results	?
Parts Appendix	?
Code	?

Introduction:

Abstract:

For our ECE 3301L final project we developed an autonomous stabilization system for a coaxial helicopter with tri-radial tilt control designed for stability.

The purpose of this project was the design of a fully autonomous stabilization system intended for systems capable of intelligent programming and navigation. This means that the design focuses mainly on the control feedback systems that control the helicopter's motors that then provide lift to increase and decrease altitude, and produce a tilt about the helicopter's central vertical axis which would be used for stabilization.

Overall Goal: Design an Autonomous Flying System:

- Sub Goal 1: Design is capable of autonomously taking off and landing
- Sub Goal 2: Design is capable of stable flight at a stationary altitude

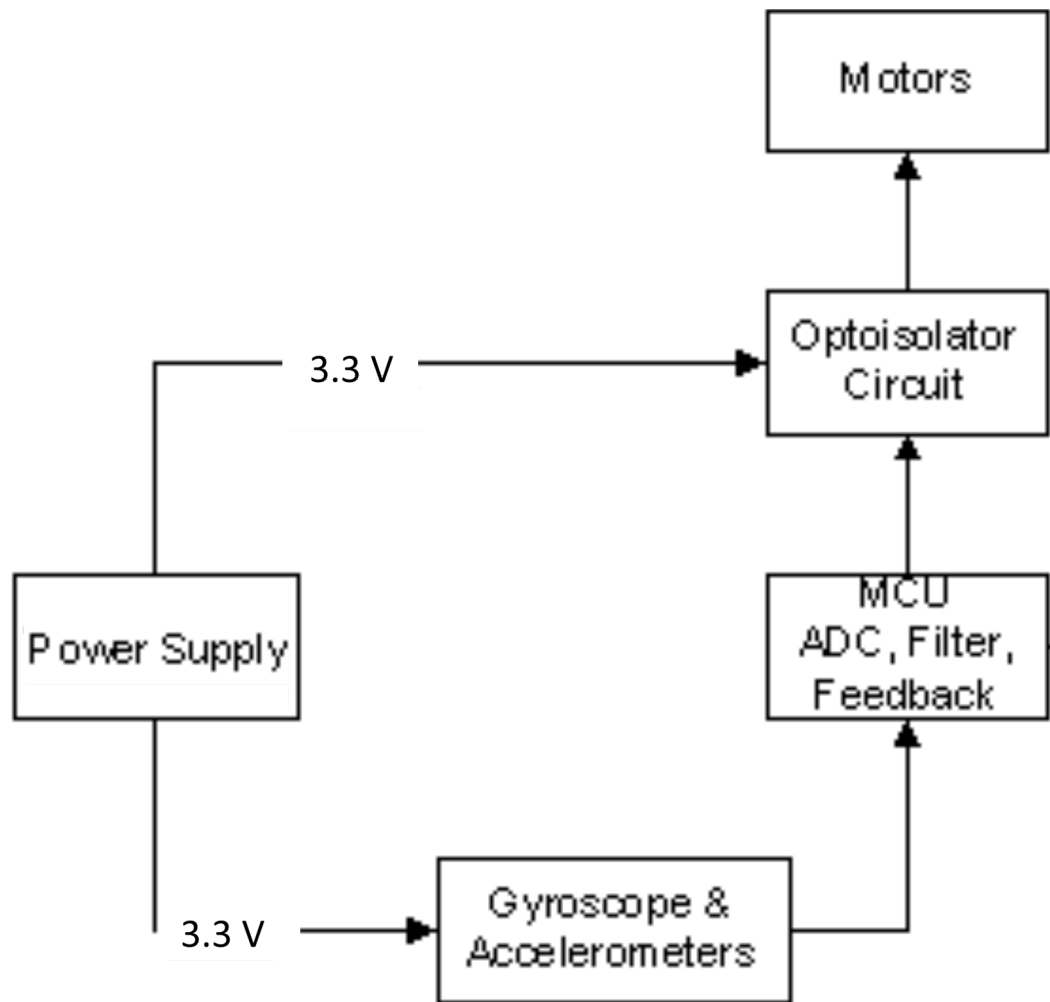
High Level Design:

Rationale:

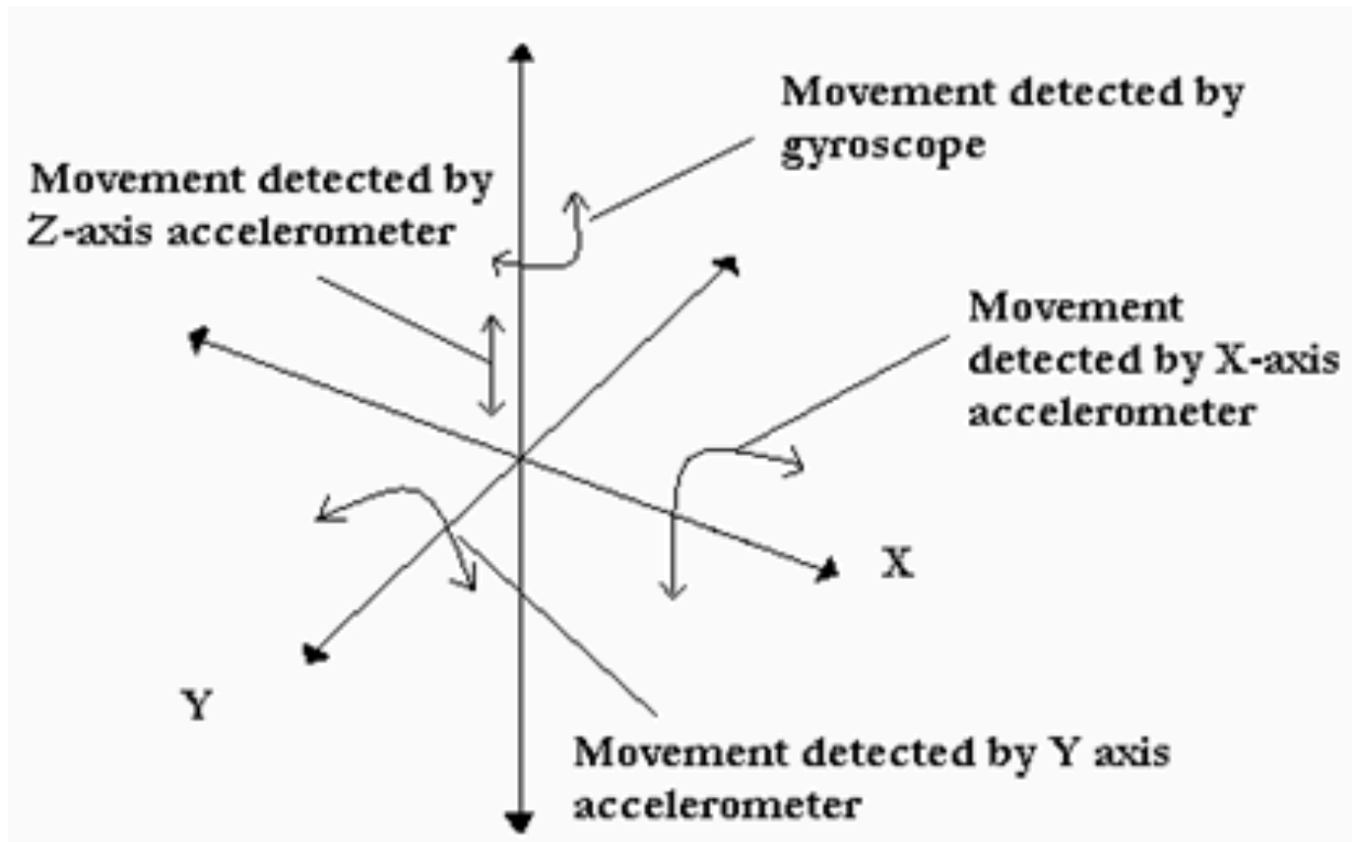
The original concept of the project is a co-axial UFO-like flying helicopter utilizing tilt control. To adhere to this design, a pre-existing co-axial rotor was outsourced, stripped down, and customized to fit the project needs. The outsourced helicopter was a Vatos RC helicopter found on Amazon. The original helicopter's chassis and circuits used to operate the RF transmission and to charge the battery were removed, leaving only the main rotors assembly to be integrated into the final project.

Structure:

A simplified diagram showing how the stabilization of the helicopter is obtained is shown below. A microcontroller was used to control the speed of the various motors to achieve lift and stabilization. This was achieved by letting the MCU output a variable PWM signal to the motors, which is adjusted according to movement detected by the combination gyroscope and accelerometer.



The movement detected by the individual sensors is shown below. Using this mechanism, a feedback loop is created to stabilize the helicopter:

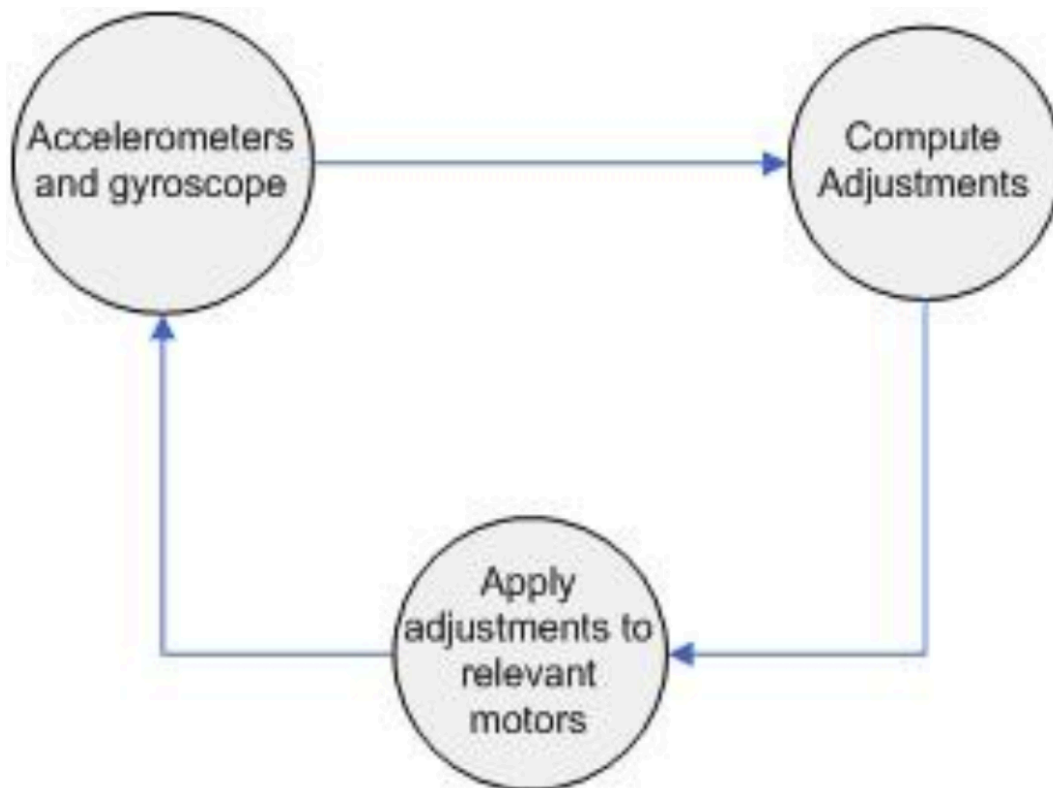


There were three major aspects of the craft that needed to be controlled using a feedback system: altitude, rotation, and orientation.

The rotation control of the craft was implemented via a control feedback system that uses a combination gyroscope/accelerometer to determine rotation and adjusts the rotor motors accordingly to keep the craft from spinning out of control.

Orientation was controlled using a basic algorithm to determine the appropriate voltage levels needed on the propellers to negate the tilt of the craft. To control altitude, the Z axis acceleration would be determined by the accelerometer and minimized by the

rotors so it stays stationary mid-air. The following diagram outlines the desired function of the above systems:



Hardware/Software Trade-Offs:

During the course of this project, there were many decisions that required compromising. For hardware, weight was the most important concern since it would require higher voltages for take off which would possibly end with burned out motors.

For software, compromises were made between noise and data resolution. Noticeable noise between the gyroscope and microcontroller was observed when the motors were

activated. Therefore software filters were implemented and the clock speed was reduced to minimize the effect of noise at the cost of less fine control over the stabilizer rotors.

Hardware:

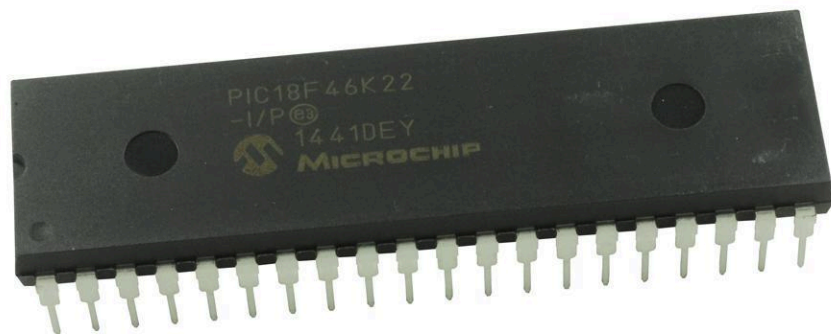
Overview:

The essential components of the hardware system were:

- Microcontroller
- Helicopter
- Stabilizer Motors
- Gyroscope/Accelerometer

Microcontroller:

The Pic18F46K22 microcontroller was mounted on a Curiosity HPC Development Board. The B and C ports were utilized for interfacing with the motors and sensors. Pins RC3 and RC4 received and output signals between the gyroscope via I2C protocol. Pins RC0, RC1, and RB5 output the PWM signals to the optocouplers that connect the motors to ground.

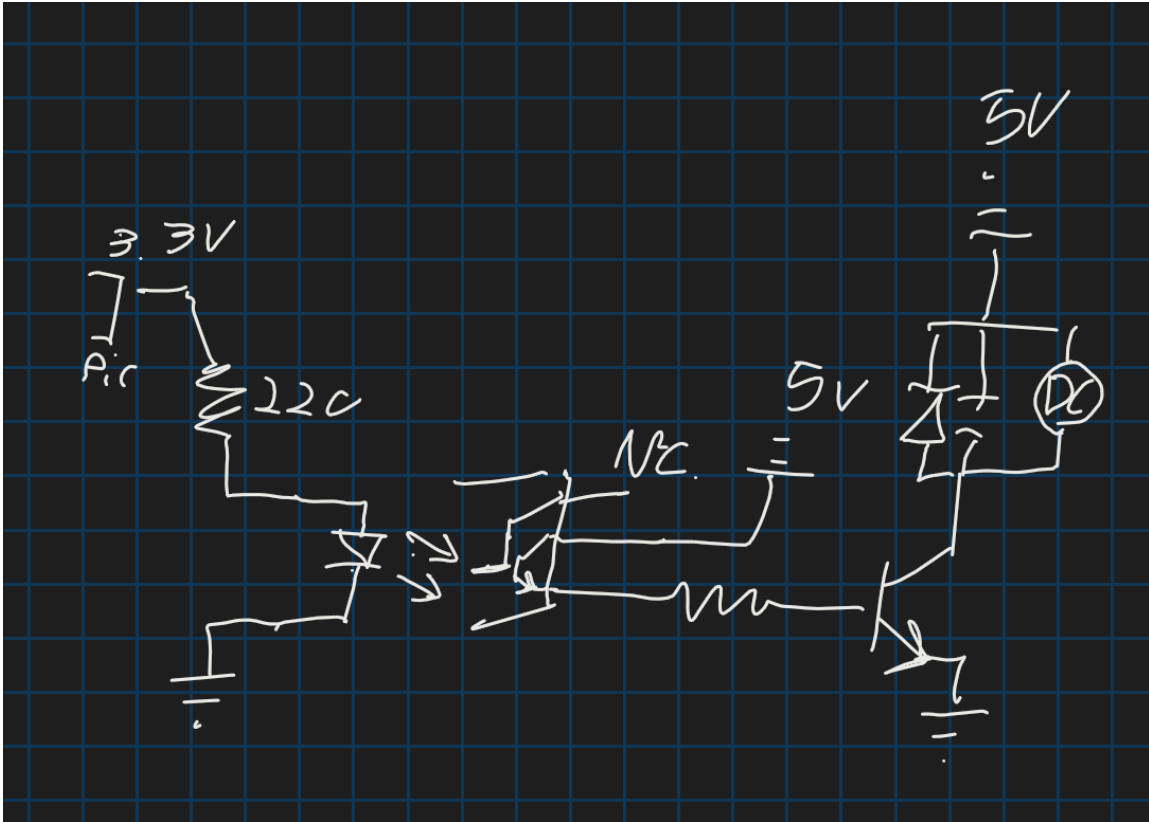


Helicopter:

A Vatos RC helicopter was purchased from Amazon to supply the main co-axial rotors and motors. The chassis, RF receiver, and lithium ion battery were removed and the gears, rotors, and motors were preserved for the design. The structure of the helicopter is such that there are two main rotors to provide lift that are individually controlled through their own motors and gear systems. For the body of the helicopter, the assembly was attached to a sheet of cardboard. Since weight was a key consideration, the microcontroller, optoisolators, and power system were moved off the helicopter and connected with wires as a detached control station, which reduced its impact on weight and would act as a tether to restrain the helicopter if needed.

Stabilizer Motors:

To interface with the total of five motors (2 main, and three for stabilizing), optocoupler circuits were implemented. The diagram for one optocoupler circuit is shown below. This circuit is duplicated four times to control the main co-axial rotor assembly and three stabilizer motors.



Gyroscope/Accelerometer:

A combination gyroscope/accelerometer was attached to the cardboard body as closely to the center as possible. To mount this onto the helicopter, a plastic enclosure was used to suspend the chip under the cardboard body. A pin header was then soldered to the chip and wires were used to connect it to the control station. A diagram displaying how it interfaces with the microcontroller is shown below:

Software:

Stability System:

The stability system consisted of 3 different control feedback systems. In the end the software was capable of having the helicopter demonstrate autonomous stabilization in all X, Y, and Z axes.

First, the gyro's level at no motion was calibrated at the start of the program. This was done when the helicopter was on the ground and still. The values generated by the gyro were then saved as the calibration point. The code would cause the microcontroller to wait for an analog conversion to complete and then check the value being received from the gyro. If this value was different from the gyro's calibration point, the stability system would infer that the helicopter was rotating in a known direction and adjust the PWM signal to each stabilizer motor to stabilize the system.

Helicopter Take-Off/Landing Routine:

The routine the helicopter takes to perform lift and stability toggled between different states controlled by a basic timing system:

GROUND: As the starting state, no stability systems are turned on, but speeds of the rotors are initialized between 0 - 30% to prepare for the TAKEOFF state.

TAKEOFF: The main co-axial rotors perform a quick increase in speed to give it the initial upward motion required to lift off the ground. This pushes PWM to the rotors to approximately 95% to give headroom for gyroscopic adjustments on the Z-axis to take place if needed. The stabilizer rotors are not utilized to ensure that the takeoff motion is as vertical as possible and that no interference will unintentionally alter the upward trajectory and cause unpredictable lateral motion.

HOVER: At this state the helicopter reaches the altitude it is ideally meant to hover at. At this point all stabilization rotors are working; if the helicopter experiences turbulence it will compensate for it by adjusting the speeds of the stabilizer rotors. This state is quickly switched on after TAKEOFF when the helicopter reaches a high enough altitude for the stability systems to be effective.

LANDING: This state is turned on after hovering long enough to demonstrate the stabilizer system. This lowers the speed of the main rotors until they reach a certain threshold speed where the helicopter begins to lower. This causes a gradual decrease in altitude until the craft gently lands.

Results:

The design did not perform to the required specifications and did not demonstrate the ability to autonomously take off. However, it did somewhat stabilize itself in all axes when manually tilted.

The helicopter was roughly able to stabilize about its x and y axes when manually tilted as the stabilizer rotors would adjust speed based on tilt. This was possibly laggy due to the microcontroller running so many different calculations and analog to digital conversions and was slowed down to 4 Hz. The wire tether caused issues as it may have added additional force and weight that interfered with lifting off.

The helicopter demonstrated capability to perform altitude calculations using the Z-axis accelerometer to theoretically maintain altitude. However, since the helicopter would not lift, practical testing was inconclusive. A very gradual increase and decrease of motor speed was programmed to ensure the system would have enough time to safely adjust for changes mid-air if it were able to lift off..

Orientation of the takeoff, balance of the craft, and the IO tether would have theoretically caused problems. These external variables were taken into account during gyroscope calibration to minimize their effects so the stability system, once activated, would be able to stabilize the craft without leaving the area in which the IO tether applied negligible force. Again, failure to lift off due to excessive weight made practical tests inconclusive

The overall results of our project were partly unsuccessful due to excessive weight and non-aerodynamic geometry of the helicopter body. However, the gyroscope control system demonstrated promising results in being able to calculate tilt and communicate with the microcontroller via I2C protocol. In the future, this system could be improved by implementing a design with less weight and better aerodynamics, and possibly integrated into a more complex aircraft such as an intelligent autonomous navigating helicopter or drone.

Parts Appendix:

PART	DESIGNATION	AMOUNT
Optoisolator	4N35	4
Gyroscope/Accelerometer	MPU-6050	1
Stabilizer Motors	8520	3
Microcontroller	Pic18F46K22	1
Curiosity HPC Development Board	DM164136	1
VATOS RC Helicopter	-	1
Wires	-	Many
Micro JST-PH 2.0mm Connectors	-	Many
Cardboard	-	1

Code:

Main:

```
#include <xc.h>
#include <stdint.h>
#include "configuration.h"
#include "pwm.h"
#include "mpu6050.h"
#include "attitude.h"

#define TIMER1_TICKS_PER_SEC  (_XTAL_FREQ / 4UL / TMR1_PRESCALE)
#define TIMER1_TICKS_PER_INT  (TIMER1_TICKS_PER_SEC / TMR1_INTERRUPT_HZ)
#define TIMER1_PRELOAD        (65536UL - TIMER1_TICKS_PER_INT)

#define TMR1H_VALUE ((uint8_t)(TIMER1_PRELOAD >> 8) & 0xFF)
#define TMR1L_VALUE ((uint8_t)(TIMER1_PRELOAD & 0xFF))
#define TMR1_PRESCALE      8
#define TMR1_INTERRUPT_HZ  400
#define ARM_DEBOUNCE_TICKS 10 // 10 * 2.5ms = 25ms debounce

// ===== GLOBALS =====
volatile uint8_t control_tick = 0; // set by Timer1 ISR

// Simple flight state machine
typedef enum {
    FLIGHT_IDLE = 0,
    FLIGHT_SPINUP,
    FLIGHT_HOVER,
    FLIGHT_FAULT,
    FLIGHT_LAND
} flight_state_t;

flight_state_t flight_state = FLIGHT_IDLE;
volatile uint8_t armed = 0;

// Desired hover throttle (YOU MUST TUNE THESE!)
const uint16_t HOVER_MAIN_THROTTLE_TARGET = 1008; // guess for hover
const uint16_t HOVER_STAB_THROTTLE_TARGET = 1008; // guess for stabilizers

// Spin-up rate (per control tick)
const uint16_t MAIN_THROTTLE_RAMP_STEP = 16; // 1 count per 2.5ms -> slow ramp
const uint16_t STAB_THROTTLE_RAMP_STEP = 16;

// ===== TIMER1: control loop @ ATT_CONTROL_HZ =====
static void Timer1_Init_ControlLoop(void)
{
    // Timer1: Fosc/4 with prescaler 8
    T1CONbits.TMR1CS = 0; // Clock = Fosc/4
    T1CONbits.T1CKPS = 0b10; // Prescaler = 8
    PIR1bits.TMR1IF = 0;
    PIE1bits.TMR1IE = 1;
    INTCONbits.PEIE = 1;
    INTCONbits.GIE = 1;
```

```

// Compute reload value
uint32_t ticks_per_sec    = (_XTAL_FREQ / 4UL) / 8UL;
uint32_t ticks_per_interrupt = (uint32_t)(ticks_per_sec / ATT_CONTROL_HZ);
uint16_t reload = (uint16_t)(65536 - ticks_per_interrupt + 1);

TMR1H = TMR1H_VALUE;
TMR1L = TMR1L_VALUE;

T1CONbits.TMR1ON = 1;
}

// ===== ISR =====
void __interrupt() isr(void)
{
    if (PIR1bits.TMR1IF) {
        TMR1H = TMR1H_VALUE;
        TMR1L = TMR1L_VALUE;
        PIR1bits.TMR1IF = 0;
        control_tick = 1;
    }
}

const float MAX_TILT = 45.0f;
// ===== MAIN =====
int main(void)
{
    // ----- Peripheral init -----
    TRISAbits.RA7=0;
    TRISAbits.RA6=0;
    TRISAbits.RA5=0;
    TRISAbits.RA4=0;
    I2C1_Init();           // MSSP1 @ 400 kHz
    MPU6050_Init();        // wake + config IMU
    PWM_Init_All(2000);    // 2 kHz PWM on all 4 channels
    Attitude_Init();       // filter + PID state
    Timer1_Init_ControlLoop();
    PORTAbits.RA7=0;
    PORTAbits.RA6=1;
    PORTAbits.RA5=0;
    PORTAbits.RA4=0;
    // Start with motors OFF
    base_throttle_main = 0;
    base_throttle_stab = 0;

    // Target: level hover
    set_roll_deg = 0.0f;
    set_pitch_deg = 0.0f;

    // Optional: pause before spin-up so you can place the heli

    flight_state = FLIGHT_SPINUP;

    mpu6050_scaled_t imu;

```

```

int count=0;

while (1) {

    if (control_tick) {
        control_tick = 0;

        MPU6050_ReadScaled(&imu);
        PORTAbits.RA7=1;

        // 2) High-level state machine: decide base throttles
        switch (flight_state) {
        case FLIGHT_IDLE:
            // Everything off

            base_throttle_main = 0;
            base_throttle_stab = 0;

            // LED ON in idle (for example)
            PORTAbits.RA5 = 1;
            PORTAbits.RA6 = 1;
            break;

        case FLIGHT_SPINUP:
            // If we somehow got here without being armed, fall back to idle

            PORTAbits.RA5 = 1;
            PORTAbits.RA6 = 0; // LED off when spinning

            // Gradually ramp up base throttles toward hover targets
            if (base_throttle_main < HOVER_MAIN_THROTTLE_TARGET)
                base_throttle_main += MAIN_THROTTLE_RAMP_STEP;

            if (base_throttle_stab < HOVER_STAB_THROTTLE_TARGET)
                base_throttle_stab += MAIN_THROTTLE_RAMP_STEP;

            // Clamp
            if (base_throttle_main > 1023) base_throttle_main = 1023;
            if (base_throttle_stab > 1023) base_throttle_stab = 1023;

            // When both reach target, go to hover
            if (base_throttle_main >= HOVER_MAIN_THROTTLE_TARGET &&
                base_throttle_stab >= HOVER_STAB_THROTTLE_TARGET) {
                flight_state = FLIGHT_HOVER;
            }
            break;

        case FLIGHT_HOVER:

            // Hold hover base throttles
            base_throttle_main = HOVER_MAIN_THROTTLE_TARGET;
            base_throttle_stab = HOVER_STAB_THROTTLE_TARGET;
            PORTAbits.RA5 = 0;
            PORTAbits.RA6 = 1;
            count+=10;
            if(count==10000) {
                flight_state=FLIGHT_LAND;
            }
            break;

```

```

case FLIGHT_FAULT:
    // Hard fault: cut power and auto-disarm
    base_throttle_main = 0;
    base_throttle_stab = 0;

    PORTAbits.RA5 = 0;
    PORTAbits.RA6=0;
    PORTAbits.RA4 = 1;
    break;

case FLIGHT_LAND:
    default:
        base_throttle_stab=0;
        base_throttle_main=MAIN_THROTTLE_RAMP_STEP*0.5;
        if (base_throttle_main<=0) {
            flight_state=FLIGHT_IDLE;
        }
        break;

    // 3) Run complementary filter + PID + mixer + PWM update
    //
    // Attitude_Update():
    // - uses imu (accel + gyro)
    // - updates att.roll_deg / att.pitch_deg
    // - uses set_roll_deg / set_pitch_deg
    // - uses base_throttle_main / base_throttle_stab
    // - writes all 4 PWM channels
    }

    Attitude_Update(&imu);
    if (count%10==0){
        if(base_throttle_main == HOVER_MAIN_THROTTLE_TARGET) {
            PORTAbits.RA4=1;
        }else{
            PORTAbits.RA4^=1;
        }
    }

    // 4) Safety check AFTER we have a fresh attitude estimate
    // If angles are insane, force FAULT for the "next" tick.
    // degrees, tune as you like
    if (att.roll_deg > MAX_TILT || att.roll_deg < -MAX_TILT ||
        att.pitch_deg > MAX_TILT || att.pitch_deg < -MAX_TILT) {
        flight_state = FLIGHT_FAULT;
    }
}
}
}

```

PWM Header:

```
#ifndef PWM_H
#define PWM_H

#include <xc.h>
#include <stdint.h>

#ifndef _XTAL_FREQ
#define _XTAL_FREQ 6400000UL // Set this to your actual clock
#endif

// Initialize all 4 PWM channels (ECCP1, CCP2, CCP3, CCP4)
// All share the same Timer2-based PWM frequency.
void PWM_Init_All(uint16_t pwm_freq_hz);

// Set duty cycle with 10-bit resolution (0..1023) for channel 1..4
// 1: ECCP1 (RC2)
// 2: CCP2 (RC1)
// 3: CCP3 (RE0)
// 4: CCP4 (RE2)
void PWM_SetDuty_10bit(uint8_t channel, uint16_t duty);

// Convenience: set duty by percentage (0.0 .. 100.0)
void PWM_SetDuty_Percent(uint8_t channel, float percent);

#endif // PWM_H
```

PWM C file:

```
#include "pwm.h"

// Internal: map 10-bit duty to CCPx registers
static void pwm_set_ccp(volatile unsigned char *ccprL,
                       volatile unsigned char *ccpCON,
                       uint16_t duty)
{
    if (duty > 1023) duty = 1023;
    uint8_t high = duty >> 2;    // upper 8 bits
    uint8_t low  = (duty & 0x03) << 4; // lower 2 bits go to <5:4>

    *ccprL = high;
    *ccpCON = (*ccpCON & 0xFC) | low; // preserve mode bits, insert duty LSBs
}

void PWM_Init_All(uint16_t pwm_freq_hz)
{
    // ----- Timer2 Setup -----
    uint8_t prescale_bits = 0b01; // Prescaler = 4
    uint32_t pr2_calc = (_XTAL_FREQ / (4UL * 4UL * pwm_freq_hz)) - 1UL;

    if (pr2_calc > 255) pr2_calc = 255;
    PR2 = (uint8_t)pr2_calc;

    T2CONbits.T2CKPS = prescale_bits;
    T2CONbits.TMR2ON = 1;

    // ----- DIGITAL Mode for CCP pins -----
    // CCP1 (RC2)
    TRISCbits.TRISC2 = 0;

    // CCP2 (RC1)
    TRISCbits.TRISC1 = 0;

    // CCP3 (RE0) -- ANALOG BY DEFAULT
    ANSELBbits.ANSB5 = 0; // !!! REQUIRED !!!
    TRISBbits.TRISB5 = 0;

    // CCP4 (RD1)
    TRISDbits.TRISD1 = 0;

    // ----- Select Timer Sources -----
    // CCP1 uses Timer2 by default (CCPTMRS0bits.C1TSEL = 0)
    // CCP2 uses Timer2 by default
    // CCP3 uses Timer2 by default
    // CCP4 uses Timer2 by default
    //
    // But we set them explicitly for safety:

    CCPTMRS0bits.C1TSEL = 0; // CCP1 -> TMR2
    CCPTMRS0bits.C2TSEL = 0; // CCP2 -> TMR2
    CCPTMRS0bits.C3TSEL = 0; // CCP3 -> TMR2
```

```

CCPTMRS1bits.C4TSEL = 0; // CCP4 -> TMR2

// ----- Enable PWM Mode on All CCPs -----
CCP1CON = 0b00001100;
CCP2CON = 0b00001100;
CCP3CON = 0b00001100;
CCP4CON = 0b00001100;

// ----- Initialize Duty Cycles -----
CCPR1L = 0;
CCPR2L = 0;
CCPR3L = 0;
CCPR4L = 0;
}

void PWM_SetDuty_10bit(uint8_t channel, uint16_t duty)
{
    if (duty > 1023) duty = 1023;

    switch (channel)
    {
        case 1: pwm_set_ccp(&CCPR1L, &CCP1CON, duty); break; // RC2
        case 2: pwm_set_ccp(&CCPR2L, &CCP2CON, duty); break; // RC1
        case 3: pwm_set_ccp(&CCPR3L, &CCP3CON, duty); break; // RE0
        case 4: pwm_set_ccp(&CCPR4L, &CCP4CON, duty); break; // RD1
        default: break;
    }
}

void PWM_SetDuty_Percent(uint8_t channel, float percent)
{
    if (percent < 0.0f) percent = 0.0f;
    if (percent > 100.0f) percent = 100.0f;

    uint16_t duty10 = (uint16_t)((percent / 100.0f) * 1023.0f);
    PWM_SetDuty_10bit(channel, duty10);
}

```

MPU6050 gyro header:

```
#ifndef MPU6050_H
#define MPU6050_H

#include <xc.h>
#include <stdint.h>

#ifndef _XTAL_FREQ
#define _XTAL_FREQ 6400000UL // same as in your project
#endif

// MPU-6050 I2C address (AD0 = 0)
#define MPU6050_ADDR_WRITE (0x68 << 1)
#define MPU6050_ADDR_READ ((0x68 << 1) | 1)

// Scaling constants for ±2g, ±250 dps
#define MPU6050_ACCEL_SENS_2G 16384.0f // LSB/g
#define MPU6050_GYRO_SENS_250 131.0f // LSB/(deg/s)

typedef struct {
    int16_t ax;
    int16_t ay;
    int16_t az;
    int16_t gx;
    int16_t gy;
    int16_t gz;
} mpu6050_raw_t;

typedef struct {
    float ax_g;
    float ay_g;
    float az_g;
    float gx_dps;
    float gy_dps;
    float gz_dps;
} mpu6050_scaled_t;

// Initialize MSSP1 for I2C master @ 400 kHz
void I2C1_Init(void);

// Initialize MPU-6050 (wake, set ranges, DLPF, samplerate)
void MPU6050_Init(void);

// Read raw accel & gyro (14-byte burst read)
void MPU6050_ReadRaw(mpu6050_raw_t *raw);

// Read scaled accel (g) & gyro (deg/s)
void MPU6050_ReadScaled(mpu6050_scaled_t *scaled);

#endif // MPU6050_H
```


MPU6050 gyro C file:

```
#include "mpu6050.h"

// ----- Low-level I2C (MSSP1) -----

void I2C1_Init(void)
{
    ANSELbits.ANSC3 = 0;
    ANSELbits.ANSC4 = 0;

    TRISCbits.TRISC3 = 1;
    TRISCbits.TRISC4 = 1;

    SSP1CON1 = 0;    // disable MSSP while configuring
    SSP1CON2 = 0;    // clear all I2C state

    // I2C Master mode, SSPEN set later:
    SSP1CON1 = 0b00101000;

    SSP1ADD = (uint8_t)((_XTAL_FREQ / (4UL * 400000UL)) - 1UL);

    SSP1STATbits.CKE = 1;
    SSP1STATbits.SMP = 0;

    SSP1CON1bits.SSPEN = 1; // finally enable MSSP
}

static inline void I2C1_WaitIdle(void)
{
    while (SSP1CON2 & 0x1F); // SEN, RSEN, PEN, RCEN, ACKEN
    while (SSP1STATbits.R_nW); // transmit in progress
}

static void I2C1_Start(void)
{
    I2C1_WaitIdle();
    SSP1CON2bits.SEN = 1;
    while (SSP1CON2bits.SEN);
}

static void I2C1_ReStart(void)
{
    I2C1_WaitIdle();
    SSP1CON2bits.RSEN = 1;
    while (SSP1CON2bits.RSEN);
}

static void I2C1_Stop(void)
{
    I2C1_WaitIdle();
    SSP1CON2bits.PEN = 1;
    while (SSP1CON2bits.PEN);
}
```

```

}

static void I2C1_Write(uint8_t data)
{
    I2C1_WaitIdle();
    SSP1BUF = data;
    while (SSP1STATbits.BF);    // wait until buffer empty
    I2C1_WaitIdle();
    // (optional) check ACK:
    // if (SSP1CON2bits.ACKSTAT) { /* no-ACK from slave */ }
}

```

```

static uint8_t I2C1_Read(uint8_t ack)
{
    uint8_t v;

    I2C1_WaitIdle();
    SSP1CON2bits.RCEN = 1;    // enable receive
    while (!SSP1STATbits.BF);    // wait for byte
    v = SSP1BUF;

    SSP1CON2bits.ACKDT = ack ? 1 : 0; // 0=ACK, 1=NACK
    SSP1CON2bits.ACKEN = 1;    // send ACK/NACK
    while (SSP1CON2bits.ACKEN);

    return v;
}

```

```

static void MPU6050_WriteReg(uint8_t reg, uint8_t val)
{
    INTCONbits.GIE = 0;

    I2C1_Start();
    I2C1_Write(MPU6050_ADDR_WRITE);
    I2C1_Write(reg);
    I2C1_Write(val);
    I2C1_Stop();

    INTCONbits.GIE = 1;
}

```

```

static uint8_t MPU6050_ReadReg(uint8_t reg)
{
    uint8_t v;
    INTCONbits.GIE = 0;

    I2C1_Start();
    I2C1_Write(MPU6050_ADDR_WRITE);
    I2C1_Write(reg);
    I2C1_ReStart();
    I2C1_Write(MPU6050_ADDR_READ);
    v = I2C1_Read(1); // NACK
    I2C1_Stop();

    INTCONbits.GIE = 1;
    return v;
}

```

```

}

void MPU6050_ReadRaw(mpu6050_raw_t *raw)
{
    uint8_t buff[14];
    INTCONbits.GIE = 0;

    I2C1_Start();
    I2C1_Write(MPU6050_ADDR_WRITE);
    I2C1_Write(0x3B);      // ACCEL_XOUT_H
    I2C1_ReStart();
    I2C1_Write(MPU6050_ADDR_READ);

    for (uint8_t i = 0; i < 13; i++) {
        buff[i] = I2C1_Read(0); // ACK
    }
    buff[13] = I2C1_Read(1);    // NACK last byte

    I2C1_Stop();

    INTCONbits.GIE = 1;

    raw->ax = ((int16_t)((buff[0] << 8) | buff[1]));
    raw->ay = ((int16_t)((buff[2] << 8) | buff[3]));
    raw->az = ((int16_t)((buff[4] << 8) | buff[5]));
    raw->gx = ((int16_t)((buff[8] << 8) | buff[9]));
    raw->gy = ((int16_t)((buff[10] << 8) | buff[11]));
    raw->gz = ((int16_t)((buff[12] << 8) | buff[13]));
}

void MPU6050_Init(void)
{
    __delay_ms(100);

    // Wake up device (PWR_MGMT_1 = 0)
    MPU6050_WriteReg(0x6B, 0x00);

    __delay_ms(10);

    // Sample rate divider (SMPLRT_DIV)
    // With DLPF on and internal 1 kHz, this sets sample rate = 1kHz/(1+7)=125 Hz
    MPU6050_WriteReg(0x19, 0x07);

    // DLPF config (CONFIG)
    // 0x03 -> DLPF ~44 Hz (good default for attitude control)
    MPU6050_WriteReg(0x1A, 0x03);

    // Gyro config (GYRO_CONFIG)
    // 0x00 -> ±250 dps
    MPU6050_WriteReg(0x1B, 0x00);

    // Accel config (ACCEL_CONFIG)
    // 0x00 -> ±2g
    MPU6050_WriteReg(0x1C, 0x00);
}

```

```

void MPU6050_ReadScaled(mpu6050_scaled_t *scaled)
{
    INTCONbits.GIE = 0;
    mpu6050_raw_t raw;
    MPU6050_ReadRaw(&raw);
    scaled->ax_g = (float)raw.ax / MPU6050_ACCEL_SENS_2G;
    scaled->ay_g = (float)raw.ay / MPU6050_ACCEL_SENS_2G;
    scaled->az_g = (float)raw.az / MPU6050_ACCEL_SENS_2G;

    scaled->gx_dps = (float)raw.gx / MPU6050_GYRO_SENS_250;
    scaled->gy_dps = (float)raw.gy / MPU6050_GYRO_SENS_250;
    scaled->gz_dps = (float)raw.gz / MPU6050_GYRO_SENS_250;
    INTCONbits.GIE = 1;
}

```

Altitude control header:

```
#ifndef ATTITUDE_H
#define ATTITUDE_H

#include <stdint.h>
#include "mpu6050.h"

// Complementary filter parameters
#define ATT_ALPHA 0.985f // gyro vs accel blending
#define ATT_CONTROL_HZ 400.0f
#define ATT_DT (1.0f / ATT_CONTROL_HZ)

// PID struct
typedef struct {
    float kp;
    float ki;
    float kd;
    float integrator;
    float last_error;
    float out_min;
    float out_max;
} pid_t;

// Public attitude state (degrees)
typedef struct {
    float roll_deg;
    float pitch_deg;

    // New:
    float roll_rate_dps; // gx (deg/sec)
    float pitch_rate_dps; // gy
    float yaw_rate_dps; // gz

    float ax; // raw accel in g
    float ay;
    float az;
} attitude_t;

// Global PIDs (one for roll, one for pitch)
extern pid_t pid_roll;
extern pid_t pid_pitch;

// Global attitude
extern attitude_t att;

// Desired setpoints (degrees)
extern float set_roll_deg;
extern float set_pitch_deg;

// Throttle settings (0..1023, 10-bit PWM)
extern uint16_t base_throttle_main; // main rotor (CCP4)
extern uint16_t base_throttle_stab; // base power for stabilizers
```

```
// Initialize PIDs and attitude state
void Attitude_Init(void);

// Run one filter + control update
// - Reads scaled IMU data (already filled by caller)
// - Updates att.roll_deg / att.pitch_deg
// - Runs PID for roll and pitch
// - Mixes into 4 motors and calls PWM_SetDuty_10bit()
void Attitude_Update(const mpu6050_scaled_t *imu);

#endif // ATTITUDE_H
```

Altitude control C file:

```
#include <math.h>
#include "attitude.h"
#include "pwm.h"

// ===== Global state and PIDs =====

pid_t pid_roll = { .kp = 3.0f, .ki = 0.5f, .kd = 0.02f,
                  .integrator = 0, .last_error = 0,
                  .out_min = -300.0f, .out_max = 300.0f };

pid_t pid_pitch = { .kp = 3.0f, .ki = 0.5f, .kd = 0.02f,
                   .integrator = 0, .last_error = 0,
                   .out_min = -300.0f, .out_max = 300.0f };

attitude_t att = { 0.0f, 0.0f };

float set_roll_deg = 0.0f;
float set_pitch_deg = 0.0f;

// Base throttles (tune these!)
uint16_t base_throttle_main = 700; // main rotor (CCP4)
uint16_t base_throttle_stab = 500; // stabilizers (CCP1/2/3)

// ===== Local helpers =====

static float pid_compute(pid_t *pid, float setpoint, float measurement, float dt)
{
    float error = setpoint - measurement;

    pid->integrator += error * dt;

    // basic anti-windup
    float integ_min = pid->out_min * 5.0f;
    float integ_max = pid->out_max * 5.0f;
    if (pid->integrator < integ_min) pid->integrator = integ_min;
    if (pid->integrator > integ_max) pid->integrator = integ_max;

    float derivative = (error - pid->last_error) / dt;
    pid->last_error = error;

    float out = pid->kp * error + pid->ki * pid->integrator + pid->kd * derivative;
    if (out < pid->out_min) out = pid->out_min;
    if (out > pid->out_max) out = pid->out_max;

    return out;
}

// Clamp integer to 0..1023
static int clamp_pwm(int v)
{
    if (v < 0) v = 0;
    if (v > 1023) v = 1023;
    return v;
}
```

```

}

void Attitude_Init(void)
{
    att.roll_deg = 0.0f;
    att.pitch_deg = 0.0f;

    pid_roll.integrator = 0.0f;
    pid_roll.last_error = 0.0f;
    pid_pitch.integrator = 0.0f;
    pid_pitch.last_error = 0.0f;
}

// ===== Attitude + control update =====
//
// Geometry of stabilizers (top view):
//
//      (front rotor, CCP3, channel 3)
//       $\theta = 0^\circ$ 
//
//      left rotor      right rotor
// (CCP1, ch1,  $\theta = 120^\circ$ ) (CCP2, ch2,  $\theta = -120^\circ$ )
//
// Main rotor (CCP4, ch4) gives overall lift.
//
// We use pitch PID and roll PID as "forces" in body axes,
// then distribute them to the three stabilizers using
// cos/sin of their angles: each rotor gets some share of
// pitch & roll correction.
//
// Convention here (can be flipped in tuning):
// - Positive roll_deg: right side down
// - Positive pitch_deg: nose up
//
const float cos0 = 1.0f;
const float sin0 = 0.0f;

//  $120^\circ$  = back-left
const float cos120 = -0.5f;
const float sin120 = 0.8660254f; //  $\sqrt{3}/2$ 

//  $240^\circ$  = back-right =  $-120^\circ$ 
const float cos240 = -0.5f;
const float sin240 = -0.8660254f; //  $-\sqrt{3}/2$ 

// How strongly pitch / roll feed the mixer (tune later)
const float MIX_PITCH_GAIN = 8.0f;
const float MIX_ROLL_GAIN = 8.0f;
int main_pwm, mot2_pwm, mot1_pwm, front_pwm;
float front_delta, m1_delta, m2_delta, tilt_comp, c_roll, c_pitch, roll_rad, pitch_rad, roll_cmd, pitch_cmd, gy, gx, accel_roll_deg, accel_pitch_deg, az, ay, ax, accel_roll_sens_deg, accel_pitch_sens_deg;
void Attitude_Update(const mpu6050_scaled_t *imu)
{
    // 1) Accelerometer-based angles (deg)
    //
    // roll from Y,Z; pitch from X,Y,Z combination

```



```

ax = imu->ax_g;
ay = imu->ay_g;
az = imu->az_g;

accel_roll_sens_deg = atan2f(ay, az) * (180.0f / (float)M_PI);
accel_pitch_sens_deg = atan2f(-ax, sqrtf(ay*ay + az*az)) * (180.0f / (float)M_PI);

// 2) Gyro rates (deg/s)
gx = imu->gy_dps;
gy = imu->gx_dps;
accel_roll_deg = accel_pitch_sens_deg;
accel_pitch_deg = accel_roll_sens_deg;

// 3) Complementary filter (gyro integration + accel correction)
att_roll_deg = ATT_ALPHA * (att_roll_deg + gx * ATT_DT)
               + (1.0f - ATT_ALPHA) * accel_roll_deg;

att_pitch_deg = ATT_ALPHA * (att_pitch_deg + gy * ATT_DT)
                + (1.0f - ATT_ALPHA) * accel_pitch_deg;

// 4) PID controllers: get correction signals (like torques)
roll_cmd = pid_compute(&pid_roll, set_roll_deg, att_roll_deg, ATT_DT);
pitch_cmd = pid_compute(&pid_pitch, set_pitch_deg, att_pitch_deg, ATT_DT);

// 5) Mix into 3 stabilizers + main rotor
//
// Physical geometry (top view, looking down, CCW positive angle):
//
// Motor 3 (CH3, CCP3):  $\theta = 0^\circ$  -> front
// Motor 1 (CH1, CCP1):  $\theta = 120^\circ$  -> back-left
// Motor 2 (CH2, CCP2):  $\theta = 240^\circ$  -> back-right ( $= -120^\circ$ )
//
// Convention (can be flipped later if needed):
// - Positive roll_deg : right side down
// - Positive pitch_deg : nose up
//
// Error = setpoint - measurement, so if nose is up (pitch_deg > 0)
// with set_pitch_deg = 0, pitch_cmd becomes negative. That will
// reduce front thrust and increase rear thrust, pushing the nose down.

// Convert roll/pitch to radians
roll_rad = att_roll_deg * (3.14159265f / 180.0f);
pitch_rad = att_pitch_deg * (3.14159265f / 180.0f);

// Compute tilt compensation factor
c_roll = cosf(roll_rad);
c_pitch = cosf(pitch_rad);

// Avoid division by zero & insane tilt
if (c_roll < 0.5f) c_roll = 0.5f;
if (c_pitch < 0.5f) c_pitch = 0.5f;

// To keep vertical component roughly constant:
tilt_comp = 1.0f / (c_roll * c_pitch);

// Optional: clamp so you don't overboost like crazy

```

```

if (tilt_comp > 1.3f) tilt_comp = 1.3f; // max +30% thrust

// Now apply to main rotor base throttle

// Front motor (Motor 3, CH3,  $\theta = 0^\circ$ )
front_delta = MIX_PITCH_GAIN * pitch_cmd * cos0
             + MIX_ROLL_GAIN * roll_cmd * sin0;

// Back-left motor (Motor 1, CH1,  $\theta = 120^\circ$ )
m1_delta = MIX_PITCH_GAIN * pitch_cmd * cos120
          + MIX_ROLL_GAIN * roll_cmd * sin120;

// Back-right motor (Motor 2, CH2,  $\theta = 240^\circ$ )
m2_delta = MIX_PITCH_GAIN * pitch_cmd * cos240
          + MIX_ROLL_GAIN * roll_cmd * sin240;

// Base throttle (stabilizers + main)
front_pwm = (int) base_throttle_stab + (int) front_delta; // CH3
mot1_pwm = (int) base_throttle_stab + (int) m1_delta; // CH1
mot2_pwm = (int) base_throttle_stab + (int) m2_delta; // CH2

// Main rotor: right now just base throttle; later add altitude control.
//main_pwm = (int) ((float) base_throttle_main * tilt_comp);
main_pwm = (int) ((float) base_throttle_main * tilt_comp);

// Clamp to 0..1023
front_pwm = clamp_pwm(front_pwm);
mot1_pwm = clamp_pwm(mot1_pwm);
mot2_pwm = clamp_pwm(mot2_pwm);
main_pwm = clamp_pwm(main_pwm);

// 6) Apply to actual PWM channels:
//
// CH1: CCP1 -> Motor 1 @ 120° (back-left)
// CH2: CCP2 -> Motor 2 @ 240° (back-right)
// CH3: CCP3 -> Motor 3 @ 0° (front)
// CH4: CCP4 -> Main rotor
PWM_SetDuty_10bit(1, (uint16_t) mot1_pwm);
PWM_SetDuty_10bit(2, (uint16_t) mot2_pwm);
PWM_SetDuty_10bit(3, (uint16_t) front_pwm);
PWM_SetDuty_10bit(4, (uint16_t) main_pwm);
}

```